10-Day Execution & Architecture Blueprint: Scoped Gravity-Time 3D Puzzle Game in Godot 4.3

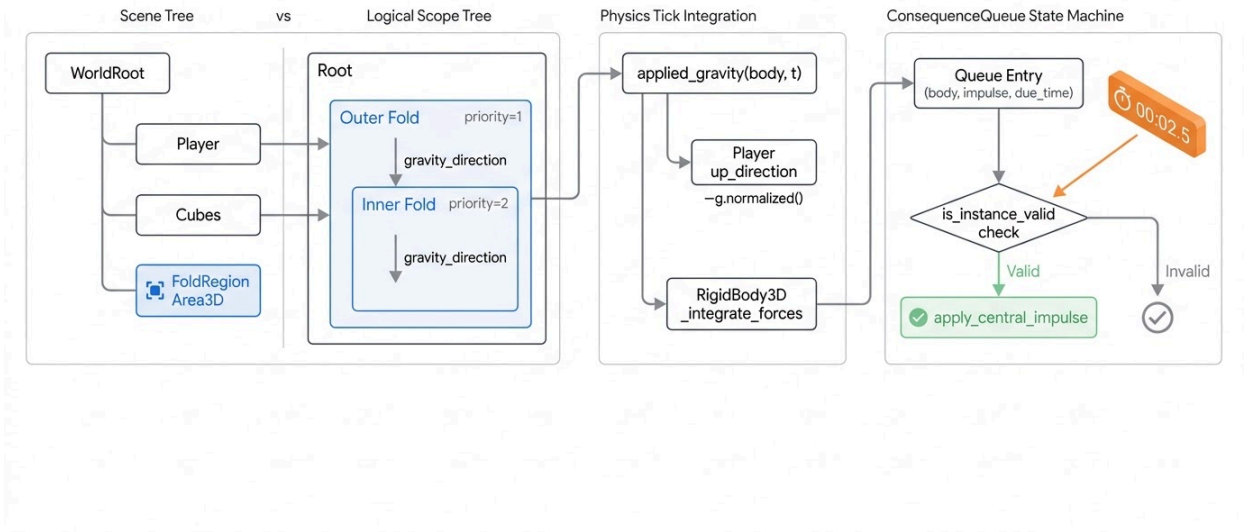
Shipping a playable, soft-lock-proof 3D puzzle course in Godot 4.3 within 10 days requires decoupling your puzzle law—bounded "folds" that locally override gravity and time and queue delayed causal impulses into the parent scope—from Godot's scene-tree transform propagation[1]. Do not make active RigidBody3D nodes scene-tree children of a moving fold node; Godot 4.3's scene tree continuously overwrites child global transforms whenever an ancestor moves, which freezes or desynchronizes rigid-body physics[2]. Instead, keep all physics bodies parented to the room root during simulation, track scope containment in an explicit FoldScope tree driven by Area3D overlaps (SPACE_OVERRIDE_REPLACE, priority = depth), freeze contained bodies (freeze = true, FREEZE_MODE_KINEMATIC) only while a fold is actively carried, and drive all cross-scope time delays through a deterministic _physics_process consequence queue[5].

To maximize your score across the Meta Global AI Developer Hackathon rubric—Impact (30%), Use of Meta AI Tooling (25%), Demo (25%), and Technical Depth & Execution (20%)[1]—keep all LLM latency out of the core puzzle loop[1]. Use free Baseten credits (muse-spark-open) for bulk GDScript generation and puzzle drafting during development[1], and spend under $3.50 of your $150 Meta Model API credits (https://api.meta.ai/v1) on an auditable build-time pipeline (muse-spark-1.3 for headless-verified room JSON and hint ladders, muse-image-1.0 + sam-3.1 for diegetic chamber signage and alpha masks, and muse-voice-transcribe-1.0 for turn-timed demo and HUD captions) plus one optional, non-blocking airlock terminal call on room completion[1]. Freeze all code at 11:59 PM ET on November 2 so November 3–4 are dedicated exclusively to recording, editing, and submitting the 3-minute demo video[1].

1. Godot 4.3 Implementation

Recommendation: Implement a single static helper FoldScope.applied_gravity(body, t, scopes) that resolves the deepest active FoldScope (Area3D) containing body and returns its directional gravity vector, while optionally configuring each fold's Area3D with gravity_space_override = Area3D.SPACE_OVERRIDE_REPLACE and priority = depth if using built-in body.get_gravity()[6]. For the per-region clock factor, reject SubViewport (which violates your no-nested-viewports rule and shares Engine.time_scale anyway)[1] and use a **Delayed-Consequence Queue** for cross-scope puzzle causality paired with a lightweight local velocity/gravity multiplier (clock_factor in 0.5 to 2.0) inside the fold[1]. Keep physics objects parented to the room root at all times except when a fold is picked up: on pickup, set body.freeze = true and reparent(fold_scope, true); on placement or break, call reparent(room_root, true), reset_physics_interpolation(), and body.freeze = false[2]. Drive delayed consequences from a pausable _physics_process accumulator that checks

is_instance_valid(body) before firing apply_central_impulse()[7].



1a. Shared applied_gravity(body, t) and Area3D Overrides

In Godot 4.3, PhysicsBody3D (inherited by both CharacterBody3D and RigidBody3D) provides Vector3 get_gravity() const, which returns the gravity vector computed from all Area3D overrides and global World3D settings affecting that body[16]. However, relying solely on Godot's internal Area3D broadphase overlap requires waiting for a physics step after node instantiation or teleportation, which complicates deterministic headless unit tests[6]. By maintaining an explicit FoldScope (Area3D) hierarchy and querying FoldScope.applied_gravity(body, t, scopes) directly while setting RigidBody3D.gravity_scale = 0.0 and Area3D.gravity_space_override = Area3D.SPACE_OVERRIDE_DISABLED (0), both CharacterBody3D and RigidBody3D share the exact same pure function without double-applying built-in gravity[6]. If you prefer to let Godot's physics server compute get_gravity() natively, set each fold's Area3D.gravity_space_override = Area3D.SPACE_OVERRIDE_REPLACE (enum value 3, which replaces default gravity and ignores lower-priority areas) and bind area.priority = get_depth()[6].

Four Godot 4.3 engine pitfalls must be guarded against when implementing scoped gravity[6]:

Godot 4.3 Mechanism	Engine Pitfall	Required Mitigation
Area3D.priority & SpaceOverride	priority defaults to 0[6]. If an outer fold and a nested inner fold both have priority = 0 and SPACE_OVERRIDE_REPLAC	Set area.priority = get_depth() (1 for outer fold, 2 for inner fold) so deeper scopes are always evaluated first before

	E (3), overlap resolution order is arbitrary[6].	World3D defaults[6].
Area3D.gravity_point	Both gravity_direction and gravity_point_center default to Vector3(0, -1, 0)[6]. If gravity_point is accidentally set to true, Vector3(0, -1, 0) acts as a point attractor 1 meter below the Area3D origin instead of a direction vector[6].	Explicitly enforce gravity_point = false in _ready() and normalize gravity_direction[6].
CharacterBody3D.up_direction & move_and_slide()	up_direction defaults to Vector3(0, 1, 0) and controls how move_and_slide() classifies floors (is_on_floor()), walls, and ceilings, as well as floor snapping[18]. If gravity inside a fold points sideways (Vector3(1, 0, 0)) and up_direction is not updated, the new floor is treated as a wall (floor_block_on_wall = true blocks walking and is_on_floor() returns false)[18].	Before calling move_and_slide(), set up_direction = -g.normalized() whenever not g.is_zero_approx() (passing Vector3.ZERO to up_direction is invalid because it is normalized internally)[18].
Physics Interpolation in 4.3	Official Godot 4.3 release notes confirm that **2D physics interpolation** shipped in 4.3, whereas **3D physics interpolation is not included in Godot 4.3** (it remained in development for 4.4)[23].	Call Node.reset_physics_interpolation() after reparenting or teleporting nodes anyway; it exists on Node in 4.3 as a safe no-op in 3D and prevents streaking if opened in Godot 4.4+[19].

GDScript

```
# res://scripts/fold_scope.gd
class_name FoldScope
extends Area3D

@export var gravity_dir: Vector3 = Vector3.DOWN
@export var gravity_mag: float = 9.8
@export var clock_factor: float = 1.0

var parent_scope: FoldScope = null
var child_scopes: Array[FoldScope] = []
var tracked_bodies: Array[PhysicsBody3D] = []

func _ready() -> void:
    monitoring = true
    gravity_point = false
    gravity_space_override = Area3D.SPACE_OVERRIDE_DISABLED # Pure manual
integration
    body_entered.connect(_on_body_entered)
    body_exited.connect(_on_body_exited)

func get_depth() -> int:
    var d := 0
    var p := parent_scope
    var visited: Array[FoldScope] = [self]
    while p != null:
        if p in visited:
            push_error("Cycle detected in FoldScope tree!")
            break
        visited.append(p)
        d += 1
        p = p.parent_scope
    return d

func set_parent_scope(new_parent: FoldScope) -> bool:
    # Invariant (i): prevent cycles so fold scopes strictly form a tree
    var cursor := new_parent
    while cursor != null:
        if cursor == self:
            return false
        cursor = cursor.parent_scope
    if parent_scope != null:
        parent_scope.child_scopes.erase(self)
```

```
    parent_scope = new_parent
    if parent_scope != null and not (self in parent_scope.child_scopes):
        parent_scope.child_scopes.append(self)
    priority = get_depth()
    return true

static func deepest_scope_for_body(body: PhysicsBody3D, scopes: Array[FoldScope]) ->
FoldScope:
    var best: FoldScope = null
    var best_depth := -1
    for s in scopes:
        if is_instance_valid(s) and body in s.tracked_bodies:
            var d := s.get_depth()
            if d > best_depth:
                best_depth = d
                best = s
    return best

static func applied_gravity(body: PhysicsBody3D, _t: float, scopes: Array[FoldScope],
default_g: Vector3 = Vector3(0, -9.8, 0)) -> Vector3:
    var scope := deepest_scope_for_body(body, scopes)
    if scope != null:
        return scope.gravity_dir.normalized() * scope.gravity_mag
    return default_g

static func applied_clock_factor(body: PhysicsBody3D, scopes: Array[FoldScope]) -> float:
    var scope := deepest_scope_for_body(body, scopes)
    return scope.clock_factor if scope != null else 1.0

func _on_body_entered(body: Node3D) -> void:
    if body is PhysicsBody3D and not (body in tracked_bodies):
        tracked_bodies.append(body)

func _on_body_exited(body: Node3D) -> void:
    if body is PhysicsBody3D:
        tracked_bodies.erase(body)
```

1b. Per-Region "Clock Factor" Comparison

Approach	Implementation	Cost & Complexity	What Breaks in Godot 4.3	Verdict

	Mechanism			
(i) Scaling delta per body	Multiply gravity/velocity by clock_factor in _physics_process (CharacterBody3D) and _integrate_forces (RigidBody3D with gravity_scale = 0.0)[7].	Low (15 lines of GDScript).	CharacterBody3D.move_and_slide() incorporates the engine's fixed delta internally[18], so you must scale velocity pre-call and unscale post-call; clock_factor > 2.5 causes collision tunneling and solver jitter at scope boundaries.	**Use only for subtle visual speed (0.5x–2.0x) inside the fold.**
(ii) Delayed-Consequence Queue	Actions/impulses triggered inside a fold of depth d and clock factor c are pushed to a central queue (body, impulse, due_time) with a 3D countdown Label3D[1].	Low (40 lines of GDScript); 100% deterministic.	Only discrete events (impulses, switches, door triggers, spawns) are delayed across the fold boundary; continuous rigid-body collisions across the boundary stay synchronous.	**RECOMMENDED PRIMARY MECHANIC.** Honest, zero engine fighting, and trivially unit-tested.
(iii) SubViewport (own_world_3	Isolate the fold in a separate SubViewport	Very High (requires portal shaders,	**Violates your frozen concept ("no**	**REJECT.**

d)	and World3D[17].	camera syncing, and proxy colliders).	**nested viewports")**[1]. Furthermore, SubViewport does not have an independent time_scale from SceneTree and blocks InputEvents by default[17].	

Here is the minimal integration showing how CharacterBody3D and RigidBody3D share applied_gravity(body, t) and applied_clock_factor(body)[7]:

```gdscript
# Player (CharacterBody3D) _physics_process snippet
func _physics_process(delta: float) -> void:
    var g := FoldScope.applied_gravity(self, Time.get_ticks_msec() * 0.001, GameWorld.active_scopes)
    var cf := FoldScope.applied_clock_factor(self, GameWorld.active_scopes)
    if not g.is_zero_approx():
        up_direction = -g.normalized()
    if not is_on_floor():
        velocity += g * (delta * cf)
    # Scale velocity temporarily for move_and_slide(), which uses unscaled delta internally
    var base_vel := velocity
    velocity = base_vel * cf
    move_and_slide()
    velocity /= maxf(cf, 0.001)

# Dynamic Cube (RigidBody3D) with gravity_scale = 0.0
func _integrate_forces(state: PhysicsDirectBodyState3D) -> void:
    if freeze:
        return
    var g := FoldScope.applied_gravity(self, Time.get_ticks_msec() * 0.001, GameWorld.active_scopes)
```

```
    var cf := FoldScope.applied_clock_factor(self, GameWorld.active_scopes)
    state.linear_velocity += g * (state.step * cf)
```

1c. Carrying, Placing, and Breaking a Fold Containing Physics Objects

Official Godot 4.3 RigidBody3D documentation explicitly warns against keeping an active RigidBody3D as a child of a moving node: *"Changing the 3D transform or linear_velocity of a RigidBody3D very often may lead to some unpredictable behaviors. This also happens when a RigidBody3D is the descendant of a constantly moving node, like another RigidBody3D, as that will cause its global transform to be set whenever its ancestor moves."*[2]. Multiple Godot 4 issues confirm that parenting an un-frozen RigidBody3D under a moving parent node causes it to freeze in place or desynchronize from its collision shape[4].

Therefore, do **not** keep active physics objects as scene-tree children of the fold during normal simulation[2]. Instead, follow a three-state reparenting protocol:

1. **While placed in the room:** Keep all RigidBody3D objects parented directly to RoomRoot and track their scope membership in fold.tracked_bodies[6].
2. **On pickup (pickup_fold):** Freeze every RigidBody3D currently inside the fold (body.freeze_mode = RigidBody3D.FREEZE_MODE_KINEMATIC, body.freeze = true) and call body.reparent(fold, true) (keep_global_transform = true) so they move rigidly with the carried fold box in front of the player camera[5].
3. **On placement (place_fold) or break (break_fold):** Iterate over the contained bodies, call body.reparent(room_root, true), call body.reset_physics_interpolation(), and set body.freeze = false[19]. When breaking a fold, reassign every contained body and child FoldScope to fold.parent_scope before removing the broken fold[1].

GDScript

```
# res://scripts/fold_manipulation.gd
class_name FoldManipulation
extends RefCounted

static func pickup_fold(fold: FoldScope, carry_anchor: Node3D) -> void:
    fold.reparent(carry_anchor, true)
    fold.transform = Transform3D.IDENTITY
    for body in fold.tracked_bodies:
        if is_instance_valid(body) and body is RigidBody3D:
            body.freeze_mode = RigidBody3D.FREEZE_MODE_KINEMATIC
            body.freeze = true
            body.reparent(fold, true)
```

```
static func place_fold(fold: FoldScope, room_root: Node3D, target_global_xform:
Transform3D) -> void:
    fold.reparent(room_root, true)
    fold.global_transform = target_global_xform
    fold.reset_physics_interpolation()
    for body in fold.tracked_bodies:
        if is_instance_valid(body) and body is RigidBody3D:
            body.reparent(room_root, true)
            body.reset_physics_interpolation()
            body.freeze = false

static func break_fold(fold: FoldScope, room_root: Node3D, all_scopes: Array[FoldScope]):
-> void:
    var parent := fold.parent_scope
    for child_fold in fold.child_scopes.duplicate():
        child_fold.set_parent_scope(parent)
    for body in fold.tracked_bodies.duplicate():
        if is_instance_valid(body):
            if body is RigidBody3D:
                body.reparent(room_root, true)
                body.reset_physics_interpolation()
                body.freeze = false
            if parent != null and not (body in parent.tracked_bodies):
                parent.tracked_bodies.append(body)
        fold.tracked_bodies.erase(body)
    all_scopes.erase(fold)
    if parent != null:
        parent.child_scopes.erase(fold)
```

1d. Delayed Consequence Queue (body, impulse, due_time)

GDScript

```
# res://scripts/consequence_queue.gd
class_name ConsequenceQueue
extends Node3D

class QueuedConsequence extends RefCounted:
    var id: int
    var body: RigidBody3D
```

```
    var impulse: Vector3
    var due_time: float
    var fired: bool = false
    var label_3d: Label3D = null

var sim_time: float = 0.0
var _next_id: int = 1
var _queue: Array[QueuedConsequence] = []

func _ready() -> void:
    process_mode = Node.PROCESS_MODE_PAUSABLE

func enqueue(body: RigidBody3D, impulse: Vector3, delay_sec: float, show_label: bool =
true) -> QueuedConsequence:
    var item := QueuedConsequence.new()
    item.id = _next_id
    _next_id += 1
    item.body = body
    item.impulse = impulse
    item.due_time = sim_time + maxf(0.0, delay_sec)
    if show_label and is_instance_valid(body):
            var lbl := Label3D.new()
            lbl.billboard = BaseMaterial3D.BILLBOARD_ENABLED
            lbl.font_size = 48
            lbl.modulate = Color(1.0, 0.56, 0.24)
            add_child(lbl)
            item.label_3d = lbl
    _queue.append(item)
    return item

func step_clock(delta: float) -> void:
    sim_time += delta
    # Iterate backwards so removal never skips an index or double-applies
    for i in range(_queue.size() - 1, -1, -1):
            var item := _queue[i]
            if item.fired:
                    _queue.remove_at(i)
                    continue
            if not is_instance_valid(item.body) or item.body.is_queued_for_deletion():
                    _cleanup_label(item)
                    _queue.remove_at(i)
                    continue
            var remaining := item.due_time - sim_time
```

```
            if is_instance_valid(item.label_3d):
                item.label_3d.global_position = item.body.global_position + Vector3(0
.2, 0)
                item.label_3d.text = "%.1fs" % maxf(0.0, remaining)
            if remaining <= 1e-5:
                item.fired = true
                _cleanup_label(item)
                queue.remove_at(i)
                item.body.freeze = false
                item.body.apply_central_impulse(item.impulse)

func _physics_process(delta: float) -> void:
    _step_clock(delta)

func _cleanup_label(item: QueuedConsequence) -> void:
    if is_instance_valid(item.label_3d):
        item.label_3d.queue_free()
        item.label_3d = null
```

This structure directly eliminates the three classic delayed-queue failure modes in Godot 4.3. First, regarding **paused physics**, SceneTree.create_timer() defaults to process_always = true, which continues ticking while SceneTree.paused == true and attempts to fire impulses while 3D physics processing is halted[8]; accumulating sim_time += delta inside a PROCESS_MODE_PAUSABLE node's _physics_process ensures the timer halts whenever the scene tree pauses[8]. Second, regarding **freed bodies**, Godot 4 Object variables do not automatically become null when freed, so checking body != null is insufficient and causes runtime errors[20]; calling @GlobalScope.is_instance_valid(item.body) alongside not item.body.is_queued_for_deletion() prevents dangling-reference crashes when a room resets[31]. Third, regarding **double-application**, iterating backward through _queue, setting item.fired = true, and removing the entry prior to calling apply_central_impulse(item.impulse) guarantees single execution even if an impulse callback triggers another scene event[7].

Evidence & Reference Links:

- Godot 4.3 Area3D Gravity & Priority Overrides: https://docs.godotengine.org/en/stable/classes/class_area3d.html[6]
- Godot 4.3 PhysicsBody3D.get_gravity(): https://docs.godotengine.org/en/4.3/classes/class_physicsbody3d.html[16]
- Godot 4.3 CharacterBody3D (up_direction, move_and_slide): https://docs.godotengine.org/en/stable/classes/class_characterbody3d.html[18]
- Godot 4.3 RigidBody3D (_integrate_forces, parent transform warning): https://docs.godotengine.org/en/4.3/classes/class_rigidbody3d.html[2]
- Godot 4.3 Release Notes (2D vs. 3D Physics Interpolation status): https://godotengine.org/releases/4.3/[23]

- Godot Node.reparent() & reset_physics_interpolation(): https://docs.godotengine.org/en/stable/classes/class_node.html[19]
- Godot @GlobalScope.is_instance_valid() & Object lifetime: https://docs.godotengine.org/en/stable/classes/class_object.html[20]

2. Headless Tests in Godot 4.3

Recommendation: Execute a standalone SceneTree-inherited script from the terminal via godot --headless --path . -s res://tests/test_runner.gd after every code change[34]. Because scripts launched with -s inherit from SceneTree (or MainLoop) and exit immediately via quit(exit_code) (0 on success, 1 on failure)[8], your AI coding assistant can verify all four invariants in a fraction of a second without opening a window or requiring GPU access[34].

CLI Invocation

```Bash
godot --headless --path . -s res://tests/test_runner.gd
```

Minimal Self-Contained Test Harness (res://tests/test_runner.gd)

```GDScript
# res://tests/test_runner.gd
extends SceneTree

var _failures: int = 0

func _init() -> void:
    _print("=== RUNNING HEADLESS INVARIANT SUITE ===")
    _test_i_scopes_form_tree()
    _test_ii_gravity_inside_fold()
    _test_iii_consequence_fires_once_on_time()
    _test_iv_breaking_fold_preserves_contents()
    if _failures == 0:
        _print("=== ALL 4 INVARIANTS PASSED ===")
        quit(0)
    else:
        _printerr("=== SUITE FAILED WITH %d ERRORS ===" % _failures)
```

```
        quit(1)

func _assert(cond: bool, msg: String) -> void:
    if not cond:
        _failures += 1
        printerr("FAIL: ", msg)
    else:
        print("PASS: ", msg)

# Invariant (i): Fold scopes form a valid tree (no cycles, accurate depth)
func test_i_scopes_form_tree() -> void:
    var outer := FoldScope.new()
    var inner := FoldScope.new()
    var leaf := FoldScope.new()
    _assert(inner.set_parent_scope(outer) == true, "(i) inner attaches to outer")
    _assert(leaf.set_parent_scope(inner) == true, "(i) leaf attaches to inner")
    _assert(outer.set_parent_scope(leaf) == false, "(i) cycle outer->leaf rejected")
    _assert(outer.get_depth() == 1 and inner.get_depth() == 2 and leaf.get_depth() == 3, "(i)
tree depths 1,2,3")
    outer.free()
    inner.free()
    leaf.free()

# Invariant (ii): Gravity inside a fold equals the deepest containing fold's gravity
func test_ii_gravity_inside_fold() -> void:
    var outer := FoldScope.new()
    outer.gravity_dir = Vector3.RIGHT
    outer.gravity_mag = 9.8
    var inner := FoldScope.new()
    inner.gravity_dir = Vector3.UP
    inner.gravity_mag = 4.9
    inner.set_parent_scope(outer)
    var body := RigidBody3D.new()
    outer.tracked_bodies.append(body)
    var scopes: Array[FoldScope] = [outer, inner]
    var g_outer := FoldScope.applied_gravity(body, 0.0, scopes)
    _assert(g_outer.is_equal_approx(Vector3(9.8, 0, 0)), "(ii) outer fold gravity overrides
world")
    inner.tracked_bodies.append(body)
    var g_inner := FoldScope.applied_gravity(body, 0.0, scopes)
    _assert(g_inner.is_equal_approx(Vector3(0, 4.9, 0)), "(ii) inner fold gravity overrides
outer")
    body.free()
```

```
    inner.free()
    outer.free()

# Invariant (iii): A queued consequence fires exactly once at the expected due_time
func test_iii_consequence_fires_once_on_time() -> void:
    var cq := ConsequenceQueue.new()
    var body := CountingRigidBody3D.new()
    var item := cq.enqueue(body, Vector3(0, 0, -10), 1.5, false)
    cq.step_clock(0.5)
    _assert(body.impulse_count == 0 and not item.fired, "(iii) not fired at t=0.5s")
    cq.step_clock(0.99)
    _assert(body.impulse_count == 0 and not item.fired, "(iii) not fired at t=1.49s")
    cq.step_clock(0.02)
    _assert(body.impulse_count == 1 and item.fired, "(iii) fired at t=1.51s (due 1.50s)")
    cq.step_clock(1.0)
    _assert(body.impulse_count == 1, "(iii) never fires a second time at t=2.51s")
    body.free()
    cq.free()

# Invariant (iv): Breaking a fold moves all contents and child folds to parent scope without
loss
func test_iv_breaking_fold_preserves_contents() -> void:
    var room := Node3D.new()
    var outer := FoldScope.new()
    var inner := FoldScope.new()
    inner.set_parent_scope(outer)
    var b1 := RigidBody3D.new()
    var b2 := RigidBody3D.new()
    room.add_child(outer)
    room.add_child(inner)
    room.add_child(b1)
    room.add_child(b2)
    inner.tracked_bodies.append_array([b1, b2])
    var scopes: Array[FoldScope] = [outer, inner]
    FoldManipulation.break_fold(inner, room, scopes)
    _assert(b1 in outer.tracked_bodies and b2 in outer.tracked_bodies, "(iv) bodies
transferred to parent scope")
    _assert(outer.tracked_bodies.size() == 2, "(iv) zero body loss on fold break")
    _assert(not (inner in scopes), "(iv) broken fold removed from active scopes")
    room.free()

class CountingRigidBody3D extends RigidBody3D:
    var impulse_count: int = 0
```

```
var last_impulse: Vector3 = Vector3.ZERO
func apply_central_impulse(impulse: Vector3 = Vector3.ZERO) -> void:
    impulse_count += 1
    last_impulse = impulse
```

Evidence & Reference Links:

- Godot Command Line Tutorial (--headless, -s, SceneTree requirement): https://docs.godotengine.org/en/latest/tutorials/editor/command_line_tutorial.html[34]
- Godot SceneTree.quit(exit_code) & Execution Order: https://docs.godotengine.org/en/stable/classes/class_scenetree.html[8]

3. Prior Art and Defensible Differentiation

Recommendation: Frame your game not as a spatial portal or global gravity-flipping puzzle, but as a **causal-latency puzzle game where carryable spatial scopes ("folds") act as kinetic capacitors**. In your 3-minute demo video, pre-empt comparisons to *Manifold Garden* and *COCOON* within the first 25 seconds by showing two objects obeying different gravity vectors and clock rates simultaneously in the same room, followed by a delayed impulse erupting out of a nested fold to open a heavy exit door after the player has already walked across the room[1].

Core Verb Comparison Across Prior Art

Game	Release / Studio	Core Verb	How Your Frozen Concept Differs
Portal	2007, Valve	**Link two non-adjacent surfaces** with portals to preserve momentum and redirect trajectories across space[37].	You do not teleport across space; you place bounded volumes inside continuous space that alter gravity and delay outgoing forces[1].
Manifold Garden	2020, William Chyr Studio	**Rotate global world gravity** across six orthogonal axes in an infinitely wrapping Escher-esque architecture[35].	Gravity never flips globally for the whole room; it is a local property of a carryable fold region alongside objects in normal gravity[1].

Maquette	2021, Graceful Decay	**Rescale objects recursively** by moving items between a miniature central dome model and the full-size surrounding courtyard[40].	Nesting changes **causal arrival time** (due_time) and local gravity rather than geometric scale[1].
Superliminal	2020, Pillow Castle	**Resize objects via forced perspective** by grabbing them at one depth and releasing them at another camera angle[41].	Relies on deterministic spatial/temporal scoping rather than camera-ray optical illusions[1].
Gravity Rush	2012, SCE Japan Studio	**Shift personal gravity** ("Gravity Shifting") to float and fall feet-first toward any aimed surface or enemy weak point[42].	Gravity is bound to a placeable spatial scope that affects any body inside it, not a superhero flight meter attached only to the player[1].
Outer Wilds	2019/2020, Mobius Digital	**Investigate a 22-minute solar-system time loop**, using knowledge of global environmental schedules to reach hidden locations[45].	Time is manipulated locally per scope by the player rather than ticking down on a single global reset clock[1].
The Sojourn	2020, Shifting Tides	**Toggle between parallel light and dark realms** and swap positions with	Folds coexist simultaneously inside one unified room rather than

		artifacts/statues to cross obstacles[46].	toggling a binary parallel-world state[1].
Patrick's Parabox	2022, Patrick Traynor	**Push boxes into and out of recursive boxes** on a discrete 2D Sokoban grid[47].	Continuous 3D rigid-body physics and real-time temporal queuing rather than discrete turn-based 2D grid steps[1].
COCOON	2023, Geometric Interactive	**Leap into and out of entire worlds contained inside carryable orbs**, nesting worlds to bring orb abilities across puzzles[36].	A fold is a translucent region inside the room—not a separate world or scene you teleport into[1].
Viewfinder	2023, Sad Owl Studios	**Place 2D photographs and images into 3D space** to overwrite level geometry and create traversable routes[47].	Preserves existing room geometry while overriding the physical laws (gravity vector and clock rate) inside a bounded volume[1].

The One Distinctive, True Sentence

> *"Instead of flipping gravity for the whole world or teleporting into separate orb-worlds, you carry and nest translucent pockets of space inside a single room—using their local gravity and clock rate to bank physical forces that erupt into the parent room on a delay."*[1]

The Two Judge Comparisons and How to Pre-Empt Them in the Demo Video

Judges watching a 3D gravity/nesting puzzle will immediately think of two touchstones: ***Manifold Garden / Gravity Rush*** (global or personal gravity flipping)[35] and ***COCOON / Portal*** (world-in-an-orb nesting or spatial teleportation)[36]. You pre-empt the *Manifold Garden* comparison in the first 10 seconds of the video by framing a static wide shot where one cube falls straight down to the floor under normal room gravity while, two meters away inside a cyan fold box, a second cube falls horizontally into a wall switch at 2.0x clock speed—visually proving that gravity and time are scoped to a region inside the room, not global[1]. You pre-empt the *COCOON* comparison at 0:52 by showing the player nest Fold B inside Fold A over an impulse

pad, trigger an impulse on a striker cube inside the nested fold, and **walk away to stand by the far exit door while a floating 4.0s countdown ticks above the cube**; when the timer hits 0.0s, the delayed impulse fires in the parent room and opens the door while the player is hands-free across the chamber[1].

Evidence & Reference Links:

- *Portal* on Steam: https://store.steampowered.com/app/400/Portal/[38]
- *Manifold Garden* on Steam: https://store.steampowered.com/app/473950/Manifold_Garden/[39]
- *Maquette* on Steam: https://store.steampowered.com/app/762840/Maquette/[40]
- *Superliminal* on Steam: https://store.steampowered.com/app/1049410/Superliminal/[41]
- *Gravity Rush* Mechanics Overview: https://en.wikipedia.org/wiki/Gravity_Rush[42]
- *Outer Wilds* on Steam: https://store.steampowered.com/app/753640/Outer_Wilds/[45]
- *The Sojourn* on Steam: https://store.steampowered.com/app/794960/The_Sojourn/[46]
- *Patrick's Parabox* on Steam: https://store.steampowered.com/app/1260520/Patricks_Parabox/[48]
- *COCOON* on Steam: https://store.steampowered.com/app/1497440/COCOON/[36]
- *Viewfinder* on Steam: https://store.steampowered.com/app/1382070/Viewfinder/[49]

4. Psychology of the First Three Minutes

Recommendation: Structure your three-room course around a strict **One-Idea-Per-Room** progression that minimizes working-memory load, uses architectural sightlines and spotlights to open a specific **curiosity gap**, and guarantees **zero-cost failure** (no lethal hazards, no manual resets required when a fold is broken)[37]. Dedicate Room 1 to *Scoped Gravity*, Room 2 to *Carrying & Breaking a Fold to Transfer Contents*, and Room 3 to the *Catch → Revelation* climax—a heavy door whose latch switch is separated by a corridor so it can only be held open by queuing a delayed impulse through a nested fold[1].

Psychological Foundations of Fair, Teachable Puzzle Rooms

In *The Gamer's Brain* (2017) and her GDC onboarding framework, Celia Hodent explains that human working memory is severely capacity-limited and suffers from *encoding deficits* when overloaded with simultaneous mechanics or text manuals[52]. Hodent divides onboarding into **Usability**—removing friction via clear physical, cognitive, and sensory affordances where *form follows function*—and **Engage-ability**, which relies on Self-Determination Theory (Deci & Ryan, 1985) to build intrinsic motivation through **competence** and **autonomy**[52]. Teaching one rule at a time and letting the player execute it immediately without HUD walls of text builds an early sense of mastery[52].

Once basic competence is established, engagement is driven by George Loewenstein's **Information Gap Theory of Curiosity** (*Psychological Bulletin*, 1994), which defines curiosity as a state of cognitively induced deprivation triggered when attention focuses on a narrow, specific gap between what one knows and wants to know[56]. Complete ignorance produces little curiosity; the designer must "prime the pump" with partial knowledge so the player can frame a concrete question[51]. Mark Brown (*Game Maker's Toolkit*) and Jonathan Blow (*Braid*, *The*

Witness) operationalize this in puzzle design through the sequence of **Assumption → Catch → Revelation**: the room invites an obvious assumption based on the rule just learned, presents a logical contradiction (the *Catch*) where the naive attempt fails by a clear margin, and rewards the player with a *Revelation* when they recombine the rule in a new way to close the curiosity gap[53].

Finally, Valve's *Portal* and *Portal 2* developer commentaries demonstrate how **predictable failure cost** and **visual telegraphing** keep experimentation frustration-free[37]. In *Portal* Test Chamber 01, Valve ensured closing portals never crushed or killed the player, pushing them safely out so they trusted the mechanic[37]. In Chamber 04, Valve added a glass barrier to block a backdoor solution (shooting a portal through the door while standing on the button) and used a warm ceiling spotlight against cool chamber lighting to draw the player's eye directly to the cube[37]. In *Portal 2*, airlock doors locked behind players between chambers so they never mistook prior rooms as part of the current puzzle[63].

Specific 3-Room Teach-By-Doing Sequence (Soft-Lock Immune)

Room	What the Player Sees on Entry	What the Player Does	What the Player Learns	Soft-Lock Prevention
Room 1: The Sideways Ledge (Scoped Gravity)	A compact chamber with an exit ledge 4m overhead (+Y, too high to jump). A glowing cyan **Fold A** (gravity = +Y UP, clock = 1.0x) rests on the floor under a warm spotlight with upward-driftin g particles[37].	Walks into Fold A and immediately falls *upward* onto the high ledge; walks across the ledge into Airlock 1. (Optional second beat: picks up Fold A from the ledge and places it to cross a short gap).	*"Gravity inside a fold points in the fold's direction, independent of the room."*[1]	No pits; stepping out of Fold A drops the player safely 4m back to the starting floor right next to Fold A[37].
Room 2: The Glass Cage (Carry & Break)	A heavy **Striker Cube** sits behind a grate that blocks the	Places **Fold A** (gravity = +X) overlapping the cube so it slides into the	*"I can carry physics objects inside a fold, and breaking a fold transfers*	Floor is gently banked toward the center plate so the cube can

	player's body but lets a placed fold overlap it. A floor pressure plate sits in the center of the room in front of the locked exit door.	fold, **picks up Fold A** (carrying the frozen cube inside it)[5], walks over the floor pressure plate, and **breaks Fold A** (Right-Click / Q). The cube drops into parent-room gravity onto the plate, opening Airlock 2[1].	*its contents into the parent room's gravity without loss."*[1]	never roll into an unreachable corner.
Room 3: The Delayed Door (Nested Fold & Queued Consequence)	**The Catch:** The final exit door sits at the end of an 8m glass corridor[37]. An **Impulse Pad** on the left wall fires a striker cube into a momentary door latch that holds the exit door open for 1.5s—closing well before the player can sprint down the 8m hall[53]. Two folds sit in the room: **Fold A** (clock = 0.5x, depth 1) and **Fold B** (clock =	**The Revelation:** Places **Fold A** over the Impulse Pad/Cube, then nests **Fold B** inside **Fold A** (creating a depth-2 scope that queues a 4.0s delayed consequence)[1]. Triggers the Impulse Pad inside the nested fold: a bright orange **4.0s floating countdown** appears above the cube[1]. The player walks	*"Anything set going inside a fold arrives in the parent world late—and nesting folds multiplies that delay so an event fires after I arrive."*[1]	The Impulse Pad is a repeatable button and the striker cube slides inside a walled guide rail that automatically returns it to the pad after 1.5s for instant retries[64].

	0.5x, depth 2).	down the 8m corridor and waits at the door. At 0.0s, the queued impulse fires in the parent room, slamming the cube into the latch and opening the final door right in front of the player[1].		

Evidence & Reference Links:

- Loewenstein (1994), *The Psychology of Curiosity: A Review and Reinterpretation*, *Psychological Bulletin*, 116(1), 75–98: https://www.cmu.edu/dietrich/sds/docs/loewenstein/PsychofCuriosity.pdf and https://doi.org/10.1037/0033-2909.116.1.75[59]
- Celia Hodent, *The Gamer's Brain* & GDC Onboarding UX: https://celiahodent.com/gamers-brain-ux-onboarding/ and https://celiahodent.com/cognitive-psychology-applied-to-user-experience-in-video-games/[54]
- Valve *Portal* & *Portal 2* Developer Commentary: https://theportalwiki.com/wiki/Portal_developer_commentary and https://combineoverwiki.net/wiki/Developer_commentary/Portal_2[37]
- Mark Brown (*Game Maker's Toolkit*), *What Makes a Good Puzzle?*: https://www.youtube.com/watch?v=zsjC6fa_YBg and https://notes.hamatti.org/gaming/puzzle-game-design[53]
- Jonathan Blow Puzzle Design Process (IndieCade / *The Witness*): https://www.gamedeveloper.com/design/indiecade-inside-jonathan-blow-s-puzzle-design-process and https://www.gamedeveloper.com/business/q-a-jonathan-blow-on-i-the-witness-i-and-the-state-of-indie-games[62]

5. Meta Model API Use That Judges Would Score

Recommendation: Build two complementary, zero-gameplay-latency integrations that are unmistakably visible in your 3-minute demo video: **(1) a Build-Time Multimodal Chamber & Diegetic Signage Pipeline** combining muse-spark-1.3 (structured JSON room configs + 3-tier hint ladders verified by your headless test runner), muse-image-1.0 ($0.01/image diegetic chamber plaques and gravity decals$), $and sam - 3.1$(2.50/1,000 images

pixel-accurate alpha masks for Godot Decal/Sprite3D nodes)[13]; and **(2) an Asynchronous Airlock "Fold Log" Telemetry Analyst + muse-voice-transcribe-1.0 Subtitle Pipeline**, which fires **one optional, non-blocking HTTP call** to muse-spark-1.3 when the final room impulse is queued (summarizing the player's fold nesting depth and delay into a 2-sentence Lab Observation on the airlock wall monitor, with instant fallback to pre-baked JSON) and uses muse-voice-transcribe-1.0 ($0.18/hour) at build time to generate turn-timed subtitles for in-game audio logs and demo narration[1]. Route all bulk GDScript coding and puzzle-parameter drafting to your free **Baseten** credits (muse-spark-open) during development so your shipped build stays 100% compliant and spends under **$3.50** of your $150 Meta Model API credits[1]. (Note: Baseten and OpenRouter documentation list the open-weight model identifier as muse-spark-open / baseten/muse-spark-open released October 15, 2026[11]; the exact date-suffixed string muse-spark-open-1026 in your brief is **UNVERIFIED** in public docs and is likely a hackathon-specific Baseten endpoint alias.)

META GLOBAL AI DEVELOPER HACKATHON 2026 · RESOURCE & LATENCY ARCHITECTURE

Meta Model API vs. Baseten Open-Weight Cost and Latency Allocation

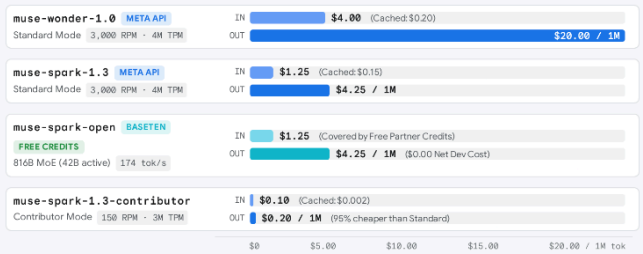
Allocating tasks across **$150 Meta Model API credits** and **free Baseten partner credits** to maximize the **25% Meta AI Tooling rubric** score while keeping multi-second LLM calls strictly outside the real-time **16.6ms (60 FPS)** puzzle loop.

TOOLING RUBRIC WEIGHT	META CREDIT BUDGET	EST. BUILD SPEND	IN-LOOP GAMEPLAY LATENCY
25%	**$150**	**<$3.50**	**0ms**
Judging score tied to creative Meta AI model integration	+ Free Baseten partner credits for open-weight inference	>97% credit buffer retained for judging & live demo	Zero synchronous LLM calls inside the 16.6ms (60 FPS) tick

LAYER 1 · TOKEN & MULTIMODAL UNIT PRICING

Text Model Pricing ($ per 1M Tokens) & Specialized Media APIs

Meta Input · Meta Output · Baseten Input · Baseten Output



SPECIALIZED MULTIMODAL & GROUNDING APIS (META MODEL API PAY-AS-YOU-GO)

Hover cards for endpoint & rate-limit specifications

muse-image-1.0	sam-3.1	muse-voice-transcribe-1.0	web_search + bg jobs
$0.01 / image	$2.50 / 1k imgs	$0.18 / hour	$2.50 / 1k queries
150 RPM · Built-in web search included	3,000 RPM · Video: $0.20/1k frames	256 streams · 16k streams/hr cap	600 RPM background job submission cap

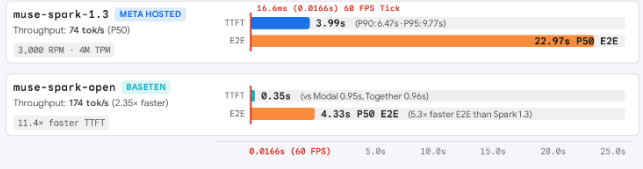
LAYER 2 · THROUGHPUT & P50 LATENCY BENCHMARK (SECONDS)

First-Token (TTFT) & End-to-End (E2E) Latency vs. 16.6ms (60 FPS) Tick

Meta P50 TTFT · Baseten P50 TTFT · P50 E2E Latency (Warning)

ARCHITECTURAL CONSTRAINT — 16.6ms (0.0166s) 60 FPS Physics Tick: Real-time puzzle physics must resolve every 0.0166s. Even Baseten's fast 0.35s first-token latency stalls 21 consecutive frames, while `muse-spark-1.3` averages 22.97s E2E (1,383 frames)—proving why all LLM reasoning belongs at build-time or behind an async transition airlock.

`Baseten TTFT: 21× Frame Tick` `Meta E2E: 1,383× Frame Tick`



LAYER 3 · EXECUTION ARCHITECTURE & CREDIT ROUTING

Three-Stage Pipeline Allocation Matrix (Dev-Time vs. Build-Time vs. Airlock Runtime)

STAGE 1 · BASETEN DEV-TIME	STAGE 2 · META BUILD-TIME	STAGE 3 · OPTIONAL AIRLOCK RUNTIME
Bulk Code Drafting & Rapid Iteration	**Verified Puzzle & Multimodal Asset Build**	**Async Level Transitions & Live Voice**
Interactive IDE & agentic repo scaffolding	Offline compilation before gameplay starts	Non-blocking calls outside 60 FPS loop
Credit Source: Free Partner Credits	Credit Source: $150 Meta API Credits	In-Loop Physics Delay: 0ms (Strict Airlock)
Meta Credit Burn: $0.00 ($1.25/$4.25 nom.)	Est. Total Spend: <$3.50 (<2.5% of pool)	Contributor Pricing: $0.10 In / $0.20 Out
Throughput / TTFT: 174 tok/s · 0.35s TTFT	Text & SAM Limits: 3,000 RPM · 4M TPM	Contributor Quota: 150 RPM · 3M TPM
E2E Turnaround: 4.33s Avg E2E	Image Rate Limit: 150 RPM ($0.01/img)	Voice ASR Stream: $0.18/hr · 256 streams
`muse-spark-open (816B MoE)`	`muse-wonder-1.0` `muse-spark-1.3` `muse-image-1.0` `sam-3.1`	`muse-spark-1.3-contributor` `muse-voice-transcribe-1.0`
Strategy: Route high-frequency coding loops and schema experiments to Baseten's 174 tok/s endpoint so **$0 of the $150 Meta API credit pool** is wasted on throwaway dev drafts.	**Strategy:** Absorbs `muse-spark-1.3`'s **22.97s E2E latency** offline while maximizing the **25% Meta AI Tooling rubric** across reasoning, sprite generation, and SAM 3.1 collision mask extraction.	**Strategy:** Any live AI interaction runs strictly in between-level loading "airlocks" (or async WebSocket streams via `wss://api.meta.ai/v1/asr/realtime`), guaranteeing an unbroken **16.6ms (60 FPS)** frame rate.

With **Muse Spark 1.3** averaging 3.99s first-token latency and 22.97s end-to-end latency, all heavy reasoning and asset generation belong at build time, while **Baseten Muse Spark Open** (174 tok/s, 0.35s latency) handles bulk dev-time drafting.

Data sources: Meta Model API, Meta Developer Documentation (Models & Pricing/Rate Limits), OpenRouter (Meta: Muse Spark 1.3 Benchmarks), OpenRouter (Meta: Muse Spark Open / Baseten Benchmarks), and Meta Global AI Developer Hackathon 2026 Overview.

Evaluation of Candidate Meta Model API Uses

Candidate Use & Meta Model(s)	Implementation Cost (Days)	Demo Visibility (in 3:00 Video)	Primary Failure Mode & Mitigation	Verdict
1. Build-Time Chamber JSON + Progressive Hint Ladders (muse-spark-1.3 / muse-wonder-1.0)	**0.5 days** (Python script calling POST /v1/responses with JSON schema + headless test check)[13].	**Medium-High** (show terminal generating room JSON + in-game [H] 3-tier hint HUD).	*Failure mode:* LLM outputs an unsolvable delay or invalid vector. *Mitigation:* Pipe output directly into godot --headless -s res://tests/test_runner.gd at build time[34].	**RECOMMENDED (#1 Core Part A).** Zero runtime risk; proves automated verification.
2. Diegetic Chamber Signage & Alpha Mask Pipeline (muse-image-1.0 + sam-3.1)	**0.5 days** (POST /v1/images/generations at $0.01/img → POST /v1/responses with sam-3.1 at $2.50/1,000 imgs)[13].	**Very High** (every room wall displays AI-generated hazard plaques & segmented decals on screen for 120+ seconds).	*Failure mode:* Rectangular image backgrounds clash with 3D wall materials. *Mitigation:* Pass each generated plaque to sam-3.1 with a short text prompt to extract a pixel-accurate alpha mask[13].	**RECOMMENDED (#1 Core Part B).** Exercises 2 distinct multimodal Meta APIs visibly.
3. Build-Time Voice-to-Subtitle & Audio Log Captioner	**0.25 days** (POST /v1/asr/transcribe at $0.18/hr	**High** (drives synchronized diegetic subtitles	*Failure mode:* Misrecognizes domain terms like	**RECOMMENDED (Support).** 2 hours of work adds a

(muse-voice-transcribe-1.0)	on recorded .wav files)[13].	in-game and burns captions into the demo video).	"FoldScope". *Mitigation:* Enable muse-voice-transcribe-1.0's built-in **contextual and keyword biasing**[14].	4th Meta model to your submission.
4. Post-Room Airlock "Fold Log" Telemetry Narrator (muse-spark-1.3 optional runtime call)	**0.5 days** (Godot HTTPRequest node firing once on final impulse queue, with local JSON fallback)[1].	**Very High** (judge sees a wall terminal print a 2-sentence analysis of the exact moves just performed in the demo).	*Failure mode:* OpenRouter reports muse-spark-1.3 average latency of 3.99s and average E2E latency of 22.97s[10]. *Mitigation:* Set reasoning.effort = "minimal", cap output at 50 tokens, trigger when the 4.0s impulse is **queued**, and fall back to pre-baked JSON after 2.5s[14].	**RECOMMENDED (#2 Core).** Gives judges a live runtime call without touching gameplay loop latency.
5. Better Unlisted Idea: Multimodal Video Playtest Critic (muse-spark-1.3 native	**0.25 days** (Upload a 20s viewport capture via POST /v1/files + POST /v1/responses to	**High** (show a 6-second split-screen in the demo where muse-spark-1.3 inspects a room video	*Failure mode:* Audio understanding on muse-spark-1.3 is not fully supported and may degrade	**Include as a 1-script build tool in tools/** to maximize the 25% Meta AI Tooling score[9].

video input)	muse-spark-1.3)[13].	clip and flags a sightline issue).	quality per official docs[14]. *Mitigation:* Strip audio (-an) or use muse-spark-1.2 when sending video clips[14].	

Smallest Honest Set of Meta Calls Visible in a 3-Minute Video & Credit Split

To capture the full 25% **Use of Meta AI Tooling** weight[1] while respecting every runtime constraint, divide your calls strictly between free Baseten partner credits for bulk dev-time drafting and Meta Model API credits for build-time assets plus one optional airlock call:

1. Free Baseten Credits (muse-spark-open — 816B total / 42B active MoE, averaging 174 tok/s and 0.35s latency on OpenRouter)[11]: Use exclusively during development (never in the shipped runtime build)[1] to draft all GDScript files, generate candidate puzzle-room parameter variations, and write headless unit tests.
2. **Meta Model API Call 1 — Build-Time Room & Hint Ladder Generator (muse-spark-1.3, POST https://api.meta.ai/v1/responses):** Script tools/build_rooms.py runs muse-spark-1.3 (reasoning: {"effort": "high"}) to output res://data/rooms.json (containing room gravity vectors, clock factors, 3-step progressive hint ladders, and fallback airlock telemetry logs) and immediately verifies solvability via godot --headless -s res://tests/test_runner.gd[13]. At $1.25/1M input tokens and $4.25/1M output tokens$ 0.10/$0.20 in Contributor mode), 20 build runs cost **under $0.10**[14].
3. **Meta Model API Call 2 — Diegetic Signage & Segmentation Pipeline (muse-image-1.0 + sam-3.1):** Script tools/build_signage.py calls POST /v1/images/generations (muse-image-1.0, $0.01/image) to render 6 chamber hazard plaques and passes each image to POST /v1/responses (sam-3.1, $2.50/1,000 images) to extract pixel-accurate alpha masks saved into res://assets/decals/[13]. Total cost: **~$0.08**[14].
4. **Meta Model API Call 3 — Voice-Over & Diegetic Subtitle Transcription (muse-voice-transcribe-1.0):** Script tools/transcribe_vo.py converts your recorded voice-over (ffmpeg -i input.mp3 -ac 1 -ar 24000 -sample_fmt s16 sample.wav) and posts it to POST https://api.meta.ai/v1/asr/transcribe with keyword biasing ("FoldScope", "clock factor", "consequence queue") to generate turn-timed subtitles in res://data/subtitles.json and demo_captions.srt[13]. At $0.18/hour, transcribing 10 takes costs **under $0.03**[14].
5. **Meta Model API Call 4 — Optional Non-Blocking Airlock "Fold Log" (muse-spark-1.3):** A single HTTPRequest node in AirlockTerminal.gd fires when the Room 3 impulse is queued, sending a compact telemetry payload ({"room": 3, "folds_nested": 2, "delay_sec": 4.0}) to POST https://api.meta.ai/v1/responses with

reasoning: {"effort": "minimal"} to print a live 2-sentence observation on the airlock wall monitor, falling back seamlessly to res://data/rooms.json if offline or if latency exceeds 2.5s[1].

Evidence & Reference Links:

- Meta Model API Overview & Endpoints (https://api.meta.ai/v1): https://dev.meta.ai/docs/overview[13]
- Meta Models Documentation (muse-spark-1.3, muse-wonder-1.0, muse-image-1.0, sam-3.1, muse-voice-transcribe-1.0): https://dev.meta.ai/docs/models[14]
- Meta Model API Pricing & Rate Limits: https://dev.meta.ai/docs/pricing-rate-limits[14]
- Muse Voice Transcribe Streaming & File Cookbook (24 kHz PCM spec): https://dev.meta.ai/resources/blog/meet-muse-voice-transcribe-streaming-speech-to-text[70]
- Baseten Hosted muse-spark-open Launch & Performance Specs: https://www.baseten.co/blog/muse-spark-open-launch/ and https://openrouter.ai/meta/muse-spark-open[11]

6. 10-Day Plan with Hard Cut Line on Nov 2

Recommendation: Enforce two hard project gates: an **October 28, 9:00 PM ET Core Physics Checkpoint** (by which FoldScope, ConsequenceQueue, and headless tests i–iv must work in Room 1, or you immediately switch to simplified fallbacks) and a **November 2, 11:59 PM ET Hard Feature Cut Line** (after which zero new mechanics or rooms are added, reserving all of November 3–4 for deterministic 60 FPS video capture, muse-voice-transcribe-1.0 caption generation, editing, and submission ahead of the November 4, 9:00 PM ET deadline)[1].

10-Day Critical Path Schedule and 180-Second Demo Rubric Mapping

- Meta Global AI Developer Hackathon (Oct 26 – Nov 4, 2026)
- Indie Developer Track ($400k Pool · $150 API Credits)
- Deadline: Nov 4, 9:00 PM ET (02:00 UTC Nov 5)

PANEL 1 · BUILD & FREEZE ROADMAP

10-Day Critical Path Execution Schedule (Oct 26 – Nov 4)

Core Godot Physics · Meta AI Pipeline · Gate / Hard Cut · Demo & Submission

DAY 1 Oct 26	Core Godot Physics	**Kickoff & Core Physics Scaffold** — 9:00 AM ET approval check, Official Rules review, Godot 2D/3D rigidbody & state init	Repo + $150 API Init
DAY 2 Oct 27	Core Godot Physics	**Player Controller & Collision Loop** — Primary movement, interaction raycasts, live workshops & Discord office hours start	Playable Prototype
DAY 3 Oct 28	PHYSICS GATE	**Oct 28 Fallback Checkpoint** — Verify core Godot physics stability; lock mechanics or trigger simplified 2D fallback scope	GATE 1: Physics Locked
OCT 28 FALLBACK GATE	Core mechanics verified before expanding to 3-room graybox & Meta AI API integration		
DAY 4 Oct 29	Core Godot Physics	**Rooms 1–2 Graybox & Puzzle Triggers** — Spatial layout, collision bounds, and deterministic hazard/objective state machines	Rooms 1–2 Graybox
DAY 5 Oct 30	Meta AI Pipeline	**Meta Model API Harness Integration** — Connect Muse Spark (text), Muse Image, and SAM 3 / Glimmer pipeline endpoints	API Bridge Active
DAY 6 Oct 31	Core Godot Physics	**3-Room Graybox Complete** — Room 3 climax layout finished; full start-to-finish walkthrough loop connected	MILESTONE: 3-Room Loop
DAY 7 Nov 1	Meta AI Pipeline	**Live AI-to-Gameplay Binding & Caching** — Bind Muse Spark / SAM 3 outputs into Room 2–3 mechanics with fallback response cache	Full AI Loop Integrated
DAY 8 Nov 2	HARD CUT LINE	**Nov 2 Hard Cut Line (Code Freeze)** — All gameplay & Meta AI pipeline code freezes at 11:59 PM ET; zero new features after cut	GATE 2: Code Freeze
NOV 2 HARD CUT LINE	45-Hour Production Buffer begins — 100% focus shifts to 180s video capture, 43s VO, and submission		
DAY 9 Nov 3	Demo & Submission	**180s Video Capture & 43s Voice-Over Edit** — Record clean 60fps 3-room footage, record 43s scripted VO, burn captions & rubric callouts	3:00 Master Cut
DAY 10 Nov 4	Demo & Submission	**Final QA & Submission by 9:00 PM ET** — Verify public video link, repo instructions, Indie Track form upload before 9:00 PM ET deadline	SUBMITTED (9:00 PM ET)

▼ Nov 3–4 (45h Window) Maps Frozen Build into 180-Second (3:00) Demo Video ▼

PANEL 2 · STORYBOARD & JUDGING ALIGNMENT

180-Second (3:00) Demo Video Beat Chart & Rubric Allocation

Impact (30%) · Meta AI Tooling (25%) · Demo Execution (25%) · Technical Depth (20%)

JUDGING RUBRIC WEIGHT → EXACT VIDEO TIME TOTAL: 100% (180S)

30% Impact	25% Meta AI Tooling	25% Demo Execution	20% Technical Depth
54s	45s	45s	36s

AUDIO ALLOCATION BUDGET — 43S VO / 137S GAMEPLAY

43s Voice-Over (24% · 108w) · 137s Pure Gameplay (76%)

0:00 – 3:00 Segmented Timeline (6 Timed Beats · Top = Rubric Axis · Bottom Strip = VO Share) — 180 Seconds Total

B1 · Impact 24s (7s VO)	B2 · Demo 25s (5s VO)	B3 · Meta AI Tooling (25%) 45s (11s VO / 34s Live AI)	B4 · Tech Depth (20%) 36s (9s VO / 27s Arch)	B5 · Demo 20s (4s VO)	B6 · Impact 30s (7s VO)
0:00	0:24	0:49	1:34	2:10	2:30	3:00 (180s)

Beat 1 · 0:00–0:24 (24s) — IMPACT · 30%
Cold Open Hook & Problem Framing
Immediate live action hook naming the target player, problem statement, and hackathon theme fit (*Build for Your World* / *Push the Frontier*).
VO: 7s (~18 words) · Gameplay: 17s

Beat 2 · 0:24–0:49 (25s) — DEMO EXEC · 25%
Room 1 Core Godot Physics Proof
Uncut playthrough of Room 1 mechanics locked at the Oct 28 Physics Gate, demonstrating responsive controls and real-time collision stability.
VO: 5s (~13 words) · Gameplay: 20s

Beat 3 · 0:49–1:34 (45s) — META AI TOOLING · 25%
Room 2 Meta AI Pipeline in Action
Showcases creative integration of Muse Spark, Muse Image, and SAM 3 / Glimmer driving dynamic level states beyond basic wrapper calls.
VO: 11s (~28 words) · Gameplay: 34s

Beat 4 · 1:34–2:10 (36s) — TECH DEPTH · 20%
Architecture & Engineering Craft
Split-screen overlay of Godot state telemetry, async Meta Model API orchestration, deterministic physics sync, and latency handling.
VO: 9s (~22 words) · Gameplay: 27s

Beat 5 · 2:10–2:30 (20s) — DEMO EXEC · 25%
Room 3 Climax (Uncut Live Loop)
High-intensity culmination of the Oct 31 3-Room Graybox build proving the game holds up live under compounded physics + AI state load.
VO: 4s (~10 words) · Gameplay: 16s

Beat 6 · 2:30–3:00 (30s) — IMPACT · 30%
Real-World Viability & Resolution
Synthesizes who benefits, why the AI-native loop matters for players and indie workflows, and closes on a cleanplayable repo title card.
VO: 7s (~17 words) · Gameplay: 23s

All gameplay and AI pipeline code freezes at the **November 2 Hard Cut Line**, leaving 45 hours before the November 4, 9:00 PM ET deadline for clean video capture, captioning, and submission.

Data sources: Meta Global AI Developer Hackathon, Reconscribe Hackathon Guide

Day-by-Day Execution Schedule (Oct 26 – Nov 4, 2026)

Date	Day	Primary Deliverable	Verification / Exit Criterion
Oct 26	Day 1	**Repo, Godot 4.3 Project & Headless Harness:** Initialize plain-text .tscn/.gd repo; implement FoldScope, ConsequenceQueue, and res://tests/test_runner.gd (use Baseten muse-spark-open for bulk scaffolding)[1].	godot --headless --path . -s res://tests/test_runner.gd exits with 0 on all 4 invariants[8].
Oct 27	Day 2	**3D Player Controller + Fold Carry/Place/Break:** Build Player.tscn (CharacterBody3D), dynamic up_direction alignment, FoldBox.tscn (Area3D + translucent BoxMesh shader), and freeze-on-pickup logic[6].	Player can walk into a fold, fall along its local gravity_dir, pick it up with a cube inside, place it, and break it cleanly.
Oct 28	Day 3	**Room 1 + Delayed Impulse Door + OCT 28 CHECKPOINT:** Build Room 1 (Scoped Gravity)	**9:00 PM ET Gate:** If 3D player camera roll or physics carry is buggy, trigger the **Oct 28 Fallbacks** below immediately.

		and prototype the Room 3 delayed impulse striker door with Label3D countdown.	
Oct 29	Day 4	**Room 2 (Carry & Break) & Room 3 (Nested Fold Door):** Graybox Rooms 2 and 3 in plain .tscn using CSGBox3D with collision enabled[75]; wire airlock transitions between Rooms 1 → 2 → 3.	Full 3-room course is playable start-to-exit using the invented law, with zero soft-locks[1].
Oct 30	Day 5	**Meta Build-Time Pipeline (tools/):** Write build_rooms.py (muse-spark-1.3 JSON + hint ladders + muse-spark-open provenance log) and build_signage.py (muse-image-1.0 + sam-3.1 decals)[13].	Generated JSON passes headless tests; 6 segmented warning decals appear on Room 1–3 walls.
Oct 31	Day 6	**Airlock Terminal + Replay Timer + HUD:** Add the non-blocking AirlockTerminal.gd (muse-spark-1.3 with 2.5s local JSON fallback), [R] instant room restart, [H] hint display, and	Completing Room 3 unlocks the Replay Timer and displays the Airlock Fold Log without any frame hitch.

		post-completion Replay Timer[1].	
Nov 1	Day 7	**Visual & Audio Telegraphing Pass:** Add warm spotlights on cubes/folds against cool chamber walls[37], directional particle streams inside folds, countdown tick SFX, and muse-spark-1.3 video playtest audit[73].	A fresh playtest run clears Rooms 1–3 in under 3 minutes without external instructions.
Nov 2	Day 8	**Bug Bash, Export Verification & HARD CUT LINE:** Fix edge-case colliders, verify clean boot from laptop, and freeze all game/AI code at **11:59 PM ET**[1].	**HARD CUT LINE:** git tag v1.0-freeze; zero feature changes permitted after 11:59 PM ET.
Nov 3	Day 9	**Demo Recording & muse-voice-transcribe-1.0 Captioning:** Capture 60 FPS lossless gameplay clips for Beats 1–6, record the 43-second voice-over, and run transcribe_vo.py (muse-voice-transcribe-1.0) to produce .srt captions[13].	Complete 3:00 video rough cut assembled with synced audio, gameplay SFX, and burned-in captions.

Nov 4	Day 10	**Final Polish, README Audit & Early Submission:** Finalize the 3:00 MP4 export, verify README provenance links, and submit by **3:00 PM ET** (6 hours ahead of the **9:00 PM ET** hard deadline)[1].	Submission confirmed on portal well before 9:00 PM ET (02:00 UTC Nov 5)[9].

Explicit October 28 Checkpoint & Three Riskiest Items with Fallbacks

Risky Item	What Fails by Oct 28, 9:00 PM ET	Concrete Fallback to Activate Immediately
1. Player Camera Roll on 90° Sideways Gravity (CharacterBody3D.up_direction)	Rotating a first-person Camera3D's local up vector by 90° while mouse-look yaw/pitch is active causes gimbal lock or motion sickness[18].	**Keep Player Horizon Upright (up_direction = Vector3.UP):** Restrict player-entered folds to vertical gravity inversions (+Y levitation lift / -Y normal) while allowing RigidBody3D cubes to fall along all 6 orthogonal axes (±X, ±Y, ±Z). The player still falls upward in Room 1 without ever rolling the camera horizon.
2. Carrying a Fold Containing Physics Cubes (RigidBody3D Collisions)	Moving a carried fold near walls pushes a child cube into static geometry or causes transform jitter on release[2].	**Holographic Ghost-Carry:** On pickup_fold, disable the cube's collision (collision_layer = 0, collision_mask = 0, freeze = true), hide its mesh, and show a miniature rotating wireframe icon inside the carried fold; on place_fold

		(validated by a ShapeCast3D so folds cannot overlap walls), snap the cube to the fold's center and re-enable collisions[16].
3. Live Airlock muse-spark-1.3 Latency (Avg 22.97s E2E)[10]	Waiting 4–23 seconds for a live API response stalls the demo pacing or fails if Wi-Fi drops[10].	**Pre-Trigger + 2.5s Hard Timeout:** Fire the HTTPRequest (reasoning.effort = "minimal", max 50 tokens) the instant the Room 3 impulse is **queued** (4.0s before the door even opens); if the response has not arrived within 2.5s of reaching the airlock monitor, display the pre-baked muse-spark-1.3 observation from res://data/rooms.json[1].

Evidence & Reference Links:

- Meta Global AI Developer Hackathon Schedule & Deadlines: https://dev.meta.ai/events/global-hackathon and https://reconscribe.com/meta-global-ai-developer-hackathon-2026/[9]
- Muse Spark 1.3 Latency Benchmarks (Avg 3.99s TTFT, Avg 22.97s E2E): https://openrouter.ai/meta/muse-spark-1.3[10]

7. Demo Script for Exactly 3:00

Recommendation: Structure your 180-second video so the first 22 seconds visually prove your physics law in a single frame without title cards, keep total spoken voice-over to **43 seconds (108 words)** across six concise sentences so the countdown ticks and physics impacts carry the pacing, and dedicate a 35-second mid-video segment (2:05–2:40) to showing your Meta Model API + Open-Weight build-time pipeline and headless test suite executing cleanly[1].

Second-by-Second Storyboard & Voice-Over Script (Total Speech: 43s / 108 Words)

Timecode	Duration	On-Screen Visual (60 FPS	Voice-Over Script (Total:	Rubric Axis Targeted

		Capture)	**43s Speech / 137s Silence & Game Audio)**	
0:00–0:22	22s	**Cold Open — The Law in One Frame:** Split-room wide shot in Room 1. Left cube drops straight down (-Y). Right cube inside a glowing cyan fold falls sideways (+X) at 2.0x speed and hits a wall pad. Player steps into a +Y fold and falls upward onto the high exit ledge.	*(0:00–0:08, 8s speech)*: *"In this game, gravity and time are local to the scope you stand in—and space itself can be folded."*[1] *(0:08–0:22: 14s pure game audio—hum of fold, upward wind rush, landing thud).*	**Demo (25%) & Impact (30%)** — Pre-empts *Manifold Garden* comparison in 8 seconds[9].
0:22–0:52	30s	**Room 2 — Carry, Place, and Break:** Player places a +X fold around a caged cube, picks up the fold (carrying the frozen cube inside), walks over the floor plate, and breaks the fold. The cube drops into	*(0:22–0:30, 8s speech)*: *"A fold is a bounded region you can carry, place, and break—transferring anything inside back to the parent world."*[1] *(0:30–0:52: 22s pure gameplay—cry*	**Technical Depth (20%) & Demo (25%)** — Shows bug-free RigidBody3D carry and scope transfer[9].

		parent gravity onto the plate, opening Airlock 2.	*stalline shatter SFX on fold break, plate click)*.	
0:52–1:42	50s	**Room 3 Climax — Nested Folds & Delayed Consequence:** Player tries the Impulse Pad normally: the far exit door opens for 1.5s and slams shut before the player is halfway down the glass hall. Player nests **Fold B** inside **Fold A** over the pad, triggers the impulse (spawning a bright orange **4.0s floating countdown**), walks down the hall to the far door, and watches the delayed impulse fire at 0.0s, blasting the heavy door open.	*(0:52–1:01, 9s speech)*: *"Anything set moving inside a fold arrives in the parent world late—and nesting folds multiplies that causal delay."*[1] *(1:01–1:42: 41s pure gameplay—rhythmic 4.0s countdown ticks, heavy impulse boom, door unlatching)*.	**Impact (30%) & Demo (25%)** — Demonstrates the Catch → Revelation and pre-empts *COCOON* comparison[9].
1:42–2:05	23s	**Airlock**	*(1:42–1:49, 7s*	**Use of Meta**

		Terminal & Live Meta API Call: Player steps into the final airlock. The wall terminal displays the muse-spark-1.3 "Fold Log" summary of the player's depth-2, 4.0s delayed solve, while synchronized subtitles (muse-voice-transcribe-1.0) appear at the bottom[13].	*speech)*: *"In the airlock—outside the physics loop—Muse Spark 1.3 analyzes our telemetry log to generate a diegetic lab observation."*[1] *(1:49–2:05: 16s terminal typing SFX).*	**AI Tooling (25%)** — Shows optional runtime call with zero gameplay latency[1].
2:05–2:40	35s	**Under the Hood — Meta Model API + Open-Weight Build Pipeline:** Fast 2-panel split screen: (Left) tools/build_rooms.py calling muse-spark-open / muse-spark-1.3 to generate room JSON and immediately running godot --headless -s res://tests/test	*(2:05–2:13, 8s speech)*: *"At build time, Muse Spark, Muse Image, and SAM 3.1 generate our room configs and segmented decals, verified by headless Godot tests."*[13] *(2:13–2:40: 27s visual terminal & mask overlay inspection).*	**Use of Meta AI Tooling (25%) & Technical Depth (20%)** — Proves multi-model orchestration and automated verification[9].

		_runner.gd (ALL 4 INVARIANTS PASSED)[12]; (Right) muse-image-1.0 generating chamber plaques and sam-3.1 extracting pixel-accurate alpha masks[13].		
2:40–3:00	20s	**Replay Timer Unlock & Clean Finish:** Back in-game, the HUD unlocks the post-completion **Replay Timer (00:00.00)**[1]. Player performs a fast 15-second speed-solve of Room 3 using a mid-air fold break, crossing the exit threshold at 2:58. Final card shows repo URL and model list.	*(2:40–2:43, 3s speech)*: *"Three rooms. One physical law. Built in ten days."* *(2:43–3:00: 17s fast replay run & final door chime)*.	**Demo (25%) & Impact (30%)** — Ends on high-competence gameplay and replayability[1].

Evidence & Reference Links:

- Meta Global AI Developer Hackathon Judging Rubric Weights: https://reconscribe.com/meta-global-ai-developer-hackathon-2026/[1]
- Muse Voice Transcribe Captioning Workflow: https://dev.meta.ai/resources/blog/meet-muse-voice-transcribe-streaming-speech-to-t

ext[70]

8. Bonus Prizes

Recommendation: Target **Best Creative Build ($5k)** as your primary bonus prize and **Best Build on Meta Open Weight Models ($10k)** as your secondary bonus target, while avoiding engineering detours for *Best Agentic Use-Case*, *Most Viral*, and *Community Vote*[1]. For a local Godot 4.3 game that cannot ship with cloud GPU servers or partner API keys, a judge will accept a build-time open-weight generation pass **only if you pair the checked-in output with an executable build script (tools/generate_with_muse_spark_open.py), a checked-in raw API response provenance log (res://data/open_weight_provenance.jsonl), a 10-second demo clip showing Muse-Spark-Open output passing godot --headless verification, and an optional local localhost runtime toggle for Meta's edge open-weight model Muse Glimmer (llama.cpp / vLLM)**[1]. (Note: Third-party event summaries confirm a **$50,000 total bonus prize pool** for standout and community awards announced on or about November 10, 2026[9], but state that the exact category values [$20k$/10k/$10k$/5k/$5k] and specific Official Rules on build-time vs. live inference were not published in the pre-kickoff overview as of October 18, 2026 and remain **UNVERIFIED** pending the Official Rules release[9].)

Bonus Prize Winnability Matrix for a Solo 10-Day Godot Game

Bonus Prize Category	Prize Value	Winnability for Solo 3D Game	Minimum Visible Artefact Required	Strategic Recommendation
Best Creative Build	$5,000 *(UNVERIFIED breakdown)*[1]	**HIGH (#1 Target)**	The 3-room playable course with simultaneous scoped gravity, floating Label3D impulse countdowns, and muse-image-1.0 + sam-3.1 diegetic chamber signage[1].	**Target directly.** Your core game and 3:00 demo already satisfy 100% of this category with zero extra scope.
Best Build on	$10,000	**MEDIUM-HIG**	Auditable build	**Target via**

Meta Open Weight Models	*(UNVERIFIED breakdown)*[1]	**H (#2 Target)**	script calling Muse-Spark-Open-816B-A42B (or local Muse Glimmer) + checked-in provenance log + 10s demo video proof of generation → headless verification[13].	**build-time pipeline** plus an optional localhost OpenAI-compatible toggle in AirlockTerminal.gd for Muse Glimmer via llama.cpp[13].
Best Agentic Use-Case	$5,000 *(UNVERIFIED breakdown)*[1]	**LOW-MEDIUM**	A multi-step autonomous tool-calling loop (muse-wonder-1.0 / muse-spark-1.3) that designs a room, runs godot --headless, reads test failures, and self-repairs the .tscn/.gd file[34].	**Secondary pitch only.** Show your build-time self-verifying loop (build_rooms.py → test_runner.gd) in Beat 5, but do not build an in-game agent.
Most Viral	$20,000 *(UNVERIFIED breakdown)*[1]	**LOW** (depends on external social media impressions)	A 20-second vertical/square clip of Beat 1 + Beat 3 (nested fold delayed door slam) posted on X/Reddit with #MetaAIHackathon.	Spend **max 15 minutes** on Nov 4 posting the first 30 seconds of your demo video; zero engineering changes.
Community	$10,000	**LOW** for a solo	High-contrast	Automatic

Vote	*(UNVERIFIED breakdown)*[1]	dev without an existing audience	thumbnail GIF of the floating countdown door blast + 1-click playable build on the submission page.	byproduct of a strong demo video; do not spend dev hours campaigning.

What a Judge Needs to See to Believe "Built on Meta Open-Weight Models"

Because a static JSON file checked into a repository could easily be hand-authored, judges evaluating **Best Build on Meta Open Weight Models** look for verifiable execution connecting Meta's open weights (Muse-Spark-Open at 816B total / 42B active parameters, or Muse Glimmer at 30B distilled under Apache 2.0) to the playable artifact[13]. You can satisfy both build-time and live-inference expectations without cloud servers through three concrete artefacts:

1. **Closed-Loop Build-Time Synthesis + Headless Godot Verification (tools/generate_chambers_open_weight.py):** Show in Beat 5 of your demo video (and in your README) a single Python command that calls Muse-Spark-Open (meta-models/Muse-Spark-Open-816B-A42B)[80] to synthesize room parameters (gravity_dir, clock_factor, delay_sec, and hint ladders), writes res://data/rooms.json, and immediately executes godot --headless --path . -s res://tests/test_runner.gd to prove the generated room satisfies all four physics invariants[34].
2. **Checked-In Cryptographic/API Provenance Log (res://data/open_weight_provenance.jsonl):** Commit the full JSONL trace of the open-weight generation pass—including timestamps, exact model ID, prompt, reasoning trace (--reasoning_parser musespark)[81], token usage, and the SHA-256 digest of the resulting res://data/rooms.json file.
3. **Optional Local Open-Weight Runtime Toggle Using Muse Glimmer (llama.cpp / vLLM):** While Muse-Spark-Open requires multi-GPU hardware (537 GB in NVFP4 or 1.65 TB in BF16)[77] and Baseten is restricted from your shipped build[1], Meta's open-weight documentation highlights **Muse Glimmer**, a multimodal model distilled from Muse Spark under an **Apache 2.0 license** specifically designed to run locally on a laptop/workstation via **llama.cpp, vLLM, SGLang, or ExecuTorch** exposing a local OpenAI-compatible endpoint[13]. By adding an optional environment/config variable LOCAL_GLIMMER_ENDPOINT = "http://127.0.0.1:8080/v1/chat/completions" in AirlockTerminal.gd (which falls back to the checked-in Muse-Spark-Open JSON when no local server is running), you can demonstrate a **100% local, zero-cloud live open-weight inference call** in your demo video while keeping the downloaded game zero-setup for judges[1].

Evidence & Reference Links:

- Meta Global AI Developer Hackathon Bonus Prize Pool & Track Breakdown: https://reconscribe.com/meta-global-ai-developer-hackathon-2026/[9]
- Muse Spark Open Documentation (816B/42B MoE, Checkpoints, Launch Partners): https://dev.meta.ai/docs/muse-spark-open and https://huggingface.co/meta-models/Muse-Spark-Open-816B-A42B[77]
- Muse Glimmer Local Open-Weight Deployment (llama.cpp, ExecuTorch, Apache 2.0): https://dev.meta.ai/docs/overview and https://dev.meta.ai/docs/models[13]

Works cited

1. gemini-prompt.md
2. RigidBody3D — Documentación de Godot Engine (4.x) en español, https://docs.godotengine.org/es/4.x/classes/class_rigidbody3d.html
3. Jolt Physics: Generic6DOFJoint3D - Unexpected movement ... - GitHub, https://github.com/godotengine/godot/issues/107461
4. Changing local transform in `_process` freezes RigidBody2D after, https://github.com/godotengine/godot/issues/120671
5. RigidBody2D — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_rigidbody2d.html
6. Area3D — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_area3d.html
7. RigidBody3D — Godot Engine (4.3) documentation in English, https://docs.godotengine.org/en/4.3/classes/class_rigidbody3d.html
8. SceneTree — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_scenetree.html
9. Meta AI Developer Hackathon 2026: $1M Prizes, Rules, Dates, https://reconscribe.com/meta-global-ai-developer-hackathon-2026/
10. Meta: Muse Spark 1.3 - API Pricing & Benchmarks - OpenRouter, https://openrouter.ai/meta/muse-spark-1.3
11. Meta: Muse Spark Open - API Pricing & Benchmarks - OpenRouter, https://openrouter.ai/meta/muse-spark-open
12. Muse Spark Open, an open-weight model for coding agents - Baseten, https://www.baseten.co/blog/muse-spark-open-launch/
13. Get started with Meta Model API and Muse Code, https://dev.meta.ai/docs/overview
14. Models - Meta Model API, https://dev.meta.ai/docs/models
15. https://dev.meta.ai/events/global-hackathon
16. PhysicsBody3D — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_physicsbody3d.html
17. SubViewport — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_subviewport.html
18. CharacterBody3D — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_characterbody3d.html

19. Node — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_node.html
20. Object — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_object.html
21. PhysicsBody3D — Godot Engine (4.3) documentation in English, https://docs.godotengine.org/en/4.3/classes/class_physicsbody3d.html
22. PhysicalBone3D — Godot Engine (4.4) documentation in English, https://docs.godotengine.org/en/4.4/classes/class_physicalbone3d.html
23. Godot 4.3, a shared effort, https://godotengine.org/releases/4.3/
24. Area3D — Documentation de Godot Engine (4.x) en français, https://docs.godotengine.org/fr/4.x/classes/class_area3d.html
25. Area3D — Documentación de Godot Engine (4.x) en español, https://docs.godotengine.org/es/4.x/classes/class_area3d.html
26. Godot 4.3 released with over 3,500 commits, including 2D physics, https://gameworldobserver.com/2024/08/19/godot-4-3-release-2d-physics-interpolation-direct3d-12
27. Viewport — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_viewport.html
28. Jolt Physics - When Reparenting a CharacterBody3D, Area3D, https://github.com/godotengine/godot/issues/113058
29. Jolt physics: Nodes reparented with keepGlobalTransform=true do, https://github.com/godotengine/godot/issues/114002
30. RigidBody3D does not follow its parent RigidBody under seemingly, https://github.com/godotengine/godot/issues/120067
31. Object — Godot Engine (3.3) documentation in English, https://docs.godotengine.org/en/3.3/classes/class_object.html
32. Object — Godot Engine (4.4) documentation in English, https://docs.godotengine.org/en/4.4/classes/class_object.html
33. @GlobalScope — Godot Engine (stable) documentation in English, https://docs.godotengine.org/en/stable/classes/class_%40globalscope.html
34. Command line tutorial - Godot Docs, https://docs.godotengine.org/en/latest/tutorials/editor/command_line_tutorial.html
35. Save 75% on Manifold Garden on Steam, https://store.steampowered.com/app/473950/?utm_source=keylol&curator_clanid=9730205
36. COCOON on Steam, https://store.steampowered.com/app/1497440/COCOON/
37. Portal developer commentary - Portal Wiki, https://theportalwiki.com/wiki/Portal_developer_commentary
38. Portal on Steam, https://store.steampowered.com/app/400/Portal/
39. Save 75% on Manifold Garden on Steam, https://store.steampowered.com/app/473950/Manifold_Garden/
40. Maquette on Steam, https://store.steampowered.com/app/762840/Maquette/
41. Superliminal on Steam, https://store.steampowered.com/app/1049410/Superliminal/

42. Gravity Rush - Wikipedia, https://en.wikipedia.org/wiki/Gravity_Rush
43. Gravity Rush - PCMag Australia, https://au.pcmag.com/sony-playstation-2ps2-games/55962/gravity-rush
44. Game Review: Gravity Rush - Vagrant Rant, https://vagrantrant.com/2012/06/28/game-review-gravity-rush/
45. Outer Wilds on Steam, https://store.steampowered.com/app/753640/Outer_Wilds/
46. Save 82% on The Sojourn on Steam, https://store.steampowered.com/app/794960/The_Sojourn/
47. Top 10 Puzzle Games for Mystery, Logic and Exploration, https://www.gametruth.com/guides/top-10-best-puzzle-games-2025/
48. Patrick's Parabox on Steam, https://store.steampowered.com/app/1260520/Patricks_Parabox/
49. Viewfinder on Steam, https://store.steampowered.com/app/1382070/Viewfinder/
50. Viewfinder on Steam, https://store.steampowered.com/app/1382070
51. How to Stimulate Curiosity | NOVA - PBS, https://www.pbs.org/wgbh/nova/article/how-to-stimulate-curiosity/
52. The Gamer's Brain Summary: Game UX Lessons for Founders, https://www.femaleswitch.com/startup-books/tpost/the-gamers-brain-summary
53. Puzzle game design : Garden of Learning by Juhis, https://notes.hamatti.org/gaming/puzzle-game-design
54. The Gamer's Brain, Part 2: UX of Onboarding and Player, https://celiahodent.com/gamers-brain-ux-onboarding/
55. Cognitive Psychology Applied to User Experience in Video Games, https://celiahodent.com/cognitive-psychology-applied-to-user-experience-in-video-games/
56. Information Gap Theory: Loewenstein's Theory of Curiosity, https://psychologyfanatic.com/information-gap-theory/
57. The Curiosity Gap Explained: Why Your Brain Demands Answers, https://chunks.app/blog/the-curiosity-gap-explained
58. The Psychology of Curiosity: A Review and Reinterpretation, https://www.researchgate.net/publication/232440476_The_Psychology_of_Curiosity_A_Review_and_Reinterpretation
59. The Psychology of Curiosity - The Encyclopedia of Abstractions, https://abstractopedia.org/references/ref-3579ffaf24db/
60. How to Stimulate Curiosity - KQED, https://www.kqed.org/mindshift/28074/how-to-stimulate-curiosity
61. How Jonathan Blow Designs a Puzzle - YouTube, https://www.youtube.com/watch?v=2zK8ItePe3Y
62. Q&A: Jonathan Blow on The Witness and the state of indie games, https://www.gamedeveloper.com/business/q-a-jonathan-blow-on-i-the-witness-i-and-the-state-of-indie-games
63. Developer commentary/Portal 2 - Combine OverWiki, https://combineoverwiki.net/wiki/Developer_commentary/Portal_2
64. What makes a good puzzle? - PC Gamer Forums,

https://forums.pcgamer.com/threads/what-makes-a-good-puzzle.1511/
65. The Psychology of Curiosity - Carnegie Mellon University, https://www.cmu.edu/dietrich/sds/docs/loewenstein/PsychofCuriosity.pdf
66. What Makes a Good Puzzle? - YouTube, https://www.youtube.com/watch?v=zsjC6fa_YBg
67. IndieCade: Inside Jonathan Blow's Puzzle Design Process, https://www.gamedeveloper.com/design/indiecade-inside-jonathan-blow-s-puzzle-design-process
68. Muse Image - Meta AI, https://dev.meta.ai/models/muse-image
69. Segment Anything Model (SAM) 3.1 - Meta AI, https://dev.meta.ai/models/sam-3-1
70. Build with Muse Voice Transcribe on Meta Model API, https://dev.meta.ai/resources/blog/meet-muse-voice-transcribe-streaming-speech-to-text
71. Muse Spark Open - Meta | API Pricing & Providers - Concentrate.ai, https://concentrate.ai/models/muse-spark-open
72. Muse Voice Transcribe - Meta AI, https://dev.meta.ai/models/muse-voice-transcribe
73. Muse Spark 1.3 - Meta AI, https://dev.meta.ai/models/muse-spark
74. Meta Model API, https://dev.meta.ai/products/meta-model-api
75. Improve the mesh asset workflow in the 3D editor #2883 - GitHub, https://github.com/godotengine/godot-proposals/issues/2883
76. Gravity Rush fans dont even get mad anymore they just watch every, https://www.reddit.com/r/gravityrush/comments/1rk9upf/gravity_rush_fans_dont_even_get_mad_anymore_they/
77. Muse Spark Open - Meta Model API, https://dev.meta.ai/docs/muse-spark-open
78. Muse Wonder | Meta, https://dev.meta.ai/models/muse-wonder
79. Muse Spark Open | Model library - Baseten, https://www.baseten.co/library/muse-spark-open/
80. meta-models/Muse-Spark-Open-816B-A42B - Hugging Face, https://huggingface.co/meta-models/Muse-Spark-Open-816B-A42B
81. Build with Muse Spark Open - Meta AI, https://dev.meta.ai/resources/blog/build-with-muse-spark-open
82. Muse Spark Open - Meta AI, https://dev.meta.ai/models/muse-spark-open